

AgentWorkShop: Node-Native Integration of Multi-Agent Teams for Industrial Supervisory Control

Anonymous Authors

Abstract—Bringing large-language-model (LLM) agents into industrial supervisory control demands associating the semantics, authority, and history of each command across heterogeneous devices, multiple agents, and changing operating recipes. We present AGENTWORKSHOP, a node-native framework in which explicit member-to-node bindings determine each agent’s tools, source-dependent checks govern agent-issued writes, and attributable records with an independent backstop make authorization, device execution, and process-level judgment inspectable as distinct events; the implementation spans five protocol drivers and lists fourteen agent engines as an inventory, not evaluated controllers. Evidence is layered across three simulated production lines—two recommissioned end to end and one as a 49-signal full line—over real protocol transports, with lifecycle probes, a four-arm governance ablation, and real-LLM closed-loop runs from prescribed cycles to objective-only optimization. The two recommissioned lines reject all predefined out-of-constraint writes—24/24, including recipe-window-only violations, and 9/9 hard-range attacks, respectively—and a fixed non-LLM controller over three seeds reaches 97.0–97.2 % of an offline grid-search reference. In one goal-driven run, the model’s first write contradicts its own prior; it reverts autonomously, estimates the local process gain from opposite-direction probes, and converges into the stated thickness band after four governed writes and one recorded revert. These are suite-coverage and single-run capability results on simulated plants, summarized per unit in Table I.

Index Terms—Industrial informatics, multi-agent systems, supervisory control, human-in-the-loop, heterogeneous protocols, large language models.

I. INTRODUCTION

LARGE language models (LLMs) are being explored for industrial diagnosis, engineering assistance, and control-program generation [1], [2], [3], [4]. Recent systems also connect agents to simulated controllers or real machines [5], [6]. For a production-line deployment, the engineering problem is therefore not whether an LLM can issue a command but how to preserve its meaning, authority, and history across heterogeneous equipment, multiple agents, and changing operating recipes.

Consider a team asked to correct film thickness within pressure and temperature limits: one member may inspect telemetry, another propose a setpoint, and a lead coordinate the task. A protocol connection alone does not identify which member may change which node, which recipe applies to the current batch, whether a human must approve, or whether the subsequent observation measures the commanded register or the process response. Implementing these associations separately for each application introduces duplicated configuration

and makes failures difficult to reconstruct; conversely, one shared interface does not by itself establish atomic execution, physical safety, or valid optimization.

We present AGENTWORKSHOP, a configurable framework for this supervisory integration problem: an industrial node combines its acquisition or setpoint interface with engineering semantics, and an explicit binding grants a deployed channel member access to that node; a line/product/recipe/run model supplies the production context shared by observations and actions. The platform core hosts agent orchestration and industrial supervision together, while engine subprocesses, optional external stores, and field endpoints retain their own boundaries. This organization reduces application-specific routing within the supported driver set without equating shared state with an end-to-end transaction.

The agent loop operates at supervisory, seconds-to-minutes timescales; regulatory control and plant-side protective functions remain in PLCs. Within the platform, proposal, admission, device execution, and outcome assessment are distinct operations. Manual bindings require an approval decision; automatic bindings omit that wait but retain the applicable execution checks; write attribution and separately triggered compensation make a failed proposal, a failed write, and an unsuccessful adjustment inspectable as different events.

The contributions are threefold:

- 1) A **node-native integration model** connecting heterogeneous acquisition and control nodes, deployed agent teams, and versioned production runs through explicit capabilities and shared identifiers (Sec. III).
- 2) An **implemented supervisory workflow** that separates team coordination, source-dependent write checks, agent-submitted judgments, physical compensation, and human decisions; its six mechanisms and the cross-cutting HITL boundary are described with their actual scopes and exceptions (Sec. IV).
- 3) A **traceable, layered evaluation** of boundary enforcement on the governed write path, integration and lifecycle operations over real protocol transports, fixed-controller loop closure, and real-LLM team operation under the same rules. Each layer yields one headline finding: all predefined out-of-constraint writes rejected (24/24, including recipe-window-only violations, and 9/9 hard-range) with no designated legal write blocked; a four-arm ablation where bypassing the soft interlock lets window-only attacks execute while hard-range rejections hold; two end-to-end recommissions and a 49-signal

full line over five protocol stacks; three fixed-controller seeds at 97.0–97.2 % of an offline grid-search reference; single-run LLM units completing a prescribed cycle and an objective-only optimization in which the model contradicts its own prior, reverts autonomously, and converges into the target band; and four exploratory campaigns exposing observation and scoring failures (Sec. V).

These contributions correspond to the evaluation’s evidence units (Sec. V, Table I): the integration model is exercised by the multi-stack and lifecycle suites, the six mechanisms by the governance and ablation probes, and the assembled workflow by the closed-loop units, each reported per unit rather than as a pooled score. The central claim is reusable supervisory integration within these tested settings, not certified safety, general industrial autonomy, or LLM-controller superiority.

II. RELATED WORK

A. Industrial Agents and Closed-Loop Applications

Agent-based distributed manufacturing control has a two-decade history [7]; what is contributed here is the binding of LLM-team membership to node capability and run context. Industrial LLM applications span industrial knowledge-graph construction [8] and diagnosis [9], PLC program generation and verification [3], [4], [10], and digital-twin configuration [11], [12]. These uses differ from supervisory setpoint execution, but not all industrial agents are advisory: natural-language interfaces have controlled machines over OPC UA [13], [6], and LLM agents have been studied in simulated fault-tolerant control [5]. InstructMPC instead uses language-derived context for disturbance prediction in model-predictive control [14]. Closest to our setting, agent-based supervision has been studied for service-oriented industrial cyber-physical systems [15], and LLM-based multi-agent systems coordinate shopfloor manufacturing tasks [16]; neither binds individual team members to specific nodes with source-dependent execution checks and line/product/recipe/run attribution. Our focus is that reusable binding, together with the governed write path and optimization records it enables, rather than a new control law or service-oriented CPS middleware.

B. Orchestration, Industrial Models, and Execution Governance

AutoGen, LangGraph, and CrewAI organize agent tasks and tool use [17], [18], [19], and MCP and A2A standardize tool and inter-agent interfaces [20], [21]; industrial integration additionally requires node semantics, driver behavior, and operating context. We use a line/product/recipe/run organization informed by ISA-95 and ISA-88 [22], [23]: line and product correspond to level-3 production work units and product definitions, run to production-performance records, and recipe to the ISA-88 master-recipe idea, with grade and operating-point semantics on continuous lines rather than a batch procedural model; we do not implement the complete procedural models of either standard. The device-twin terminology used throughout presumes manufacturing digital twins in which assets carry structured semantic descriptions [24], [25]; the node semantic

card is analogous to an Asset Administration Shell submodel [26], supplying the engineering quantities needed at the tool boundary. A goal-driven unit’s observe–hypothesize–write–re-observe cycle likewise resembles autonomous chemical research with LLMs [27], which closes the loop over physical experiments; our writes are governed industrial setpoints rather than self-directed experiment planning.

The ingredients of a governed write—node-scoped permission, range validation, approval, and an audit trail—have counterparts in mature DCS/SCADA/MES practice, typically enforced as role- and application-level controls under IEC 62443-3-3 [28]; LLM-mediated writes additionally require member-instance-level binding, semantic linkage between a write and the verdict or optimization record that produced it, and attribution across engine boundaries. Hard action shielding and control-barrier methods enforce formally specified admissible actions [29], [30]; the rollback manager and rule-based backstop are engineering instances of run-time assurance and simplex-style architectures [31], in the spirit of the safe-control literature [32]—narrower implementation-level checks, not substitutes for those methods or for certified plant protection. Human supervision also requires a clear division of responsibility, not merely an approval dialog [33], [34], and the manual/auto binding modes engage trust calibration [35].

C. Evaluation of Tools and Industrial Control

AgentBench and consequential tool-use benchmarks evaluate agent behavior and policy compliance [36], [37], [38]. Industrial datasets such as TEP and SWaT support process-monitoring research [39], [40], but replay is not evidence of successful physical actuation, so we separate source checks, mock-driver probes, software-protocol integration, fixed-controller optimization, and exploratory LLM runs; no external-framework ranking, dataset-replay result, or physical-device validation is claimed. The threat model of indirect prompt injection [41]—a compromised agent writing through legitimate tools—is what the write-side checks of the governed write path are designed to contain, although our evaluation includes no adversarial LLM probes.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP integrates multi-agent reasoning with industrial supervision around a common representation of production assets and operating context. Its organizing object is the node: an acquisition or control endpoint with engineering semantics, device connections, line membership, and agent bindings. A channel organizes the agents that interpret these endpoints, controllers mediate plant interactions, and Fig. 1 locates these components and execution boundaries.

A. A Shared Supervisory Core

The Node.js core hosts channel routing, task scheduling, industrial services, protocol adapters, and platform interfaces: REST and WebSocket endpoints serve operator applications, MCP and A2A interfaces support external integration, and the plugin host extends platform capabilities. These surfaces reuse

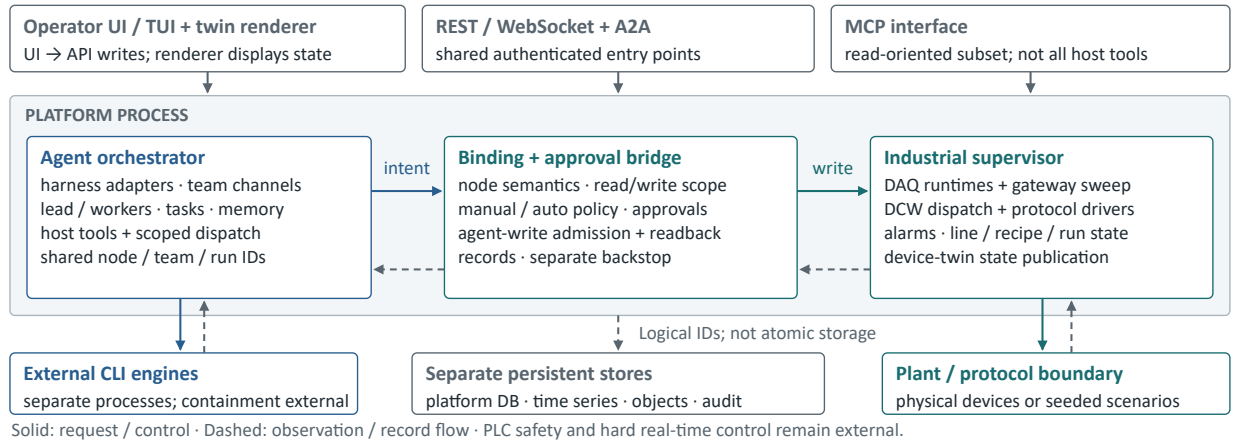


Fig. 1. System architecture and execution boundaries. The co-resident platform core connects agent channels, semantic nodes, industrial controllers, and operator services. CLI/OMP engines and storage services extend beyond that process. Shared identifiers support attribution across the logical data model; they do not imply a distributed transaction or atomic physical actuation.

platform objects, but their exposed operations and permission checks differ: an agent's industrial authority derives from its node bindings, not from channel access or from the mere availability of a protocol endpoint.

Co-residency reduces the need to synchronize duplicate nodes, bindings, and runs across separately assembled applications, but does not place the deployment in one process: CLI engines and the platform's bundled OMP agent harness can execute in spawned processes, TimescaleDB and MinIO are external, and connection establishment, queued protocol operations, and readback introduce asynchronous boundaries. Fast regulation and protection therefore remain with the plant controller and its safety equipment.

B. Reasoning, Plant Interaction, and Operational Context

On the reasoning side, a channel combines a lead, worker instances, a task tree, messages, and retrievable memory. The scheduler couples model-based supervision to deterministic dispatch and progress tracking, and engine adapters allow different reasoning implementations to participate without changing a bound node's identity or the controller interface. This separates task organization from device authority: assigning a worker a task does not grant permission to execute the associated industrial action.

On the plant side, acquisition nodes turn protocol readings into engineering quantities, control nodes translate engineering setpoints into device commands and readback results, per-node runtimes manage timing and in-flight operations, and controller services supply gateway state, recipe context, and write bookkeeping. Nodes and devices have many-to-many configuration associations: one device can supply several semantic nodes, and a node can reference several devices. Association is distinct from execution routing: a control-node write invokes its configured driver and connection settings rather than iterating over its associated devices, so the interface follows process meaning without implying multi-device actuation by one write.

The bridge between these sides consists of semantic cards, instance-specific bindings, industrial tools, and source-

dependent controller checks; cards expose units, ranges, current values, and relevant process context. Bindings determine whether a channel member can request an operation and whether human approval is needed. Controllers then apply the checks appropriate to that request's origin: semantic guidance helps an agent propose a sensible action, whereas executable checks determine which requests reach a driver. Prompt guidance, such as the suggested step, is not an enforced interlock.

C. Logical Continuity Across a Production Run

Lines, products, recipes, and runs provide the context in which measurements and interventions become comparable; their identifiers connect platform state, time series, image frames, operational events, and optimization records across heterogeneous stores—SQLite and file-backed repositories for platform state, TimescaleDB for measurements, MinIO for objects. Together these constitute one logical data model, not one transaction domain: the journal records attributable changes and judgments, but is not a cryptographically signed ledger, and bookkeeping failure does not necessarily undo an already completed write.

An end-to-end interaction begins when an engineer connects devices, defines nodes, associates production context, and binds channel members; starting a line establishes an active run and initiates its parameter writes. Agents inspect observations, exchange evidence, and propose adjustments through their bound tools; accepted requests proceed to device communication and readback, and subsequent observations inform an agent-supplied judgment and an independent rule-based backstop; recovery may require a further physical write, not merely a recipe redefinition.

Operators participate through approvals, line operations, and monitoring. The device-twin renderer visualizes state without performing protocol I/O, but controls in the Town operator web interface can request API writes, so visualization is separated from device transport, not from all operator actuation. Section IV details the six mechanisms and the cross-cutting HITL boundary, and marks where their guarantees end.

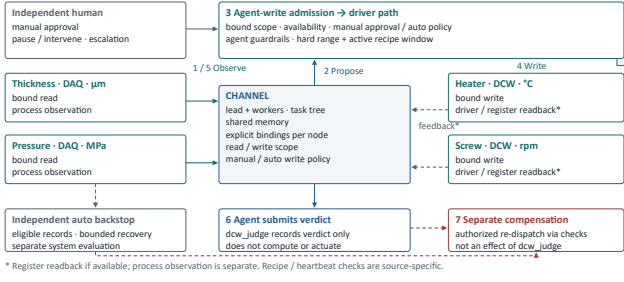


Fig. 2. The six mechanisms in their organizational setting and closed loop: a channel coordinates agents that observe bound nodes, propose setpoints, and pass admission checks before write and readback; judgment, an independent backstop, and separate compensation complete the cycle, with human intervention at approval and escalation. `dcw_judge` records a verdict only: it neither computes the conclusion from telemetry nor executes any action.

IV. CORE MECHANISMS

The six core mechanisms below are node connection and creation (Section IV-A), the agent channel (Section IV-B), node control execution (Section IV-C), production-line operations (Section IV-D), agent closed-loop control (Section IV-E), and recipe management and rollback (Section IV-F), together with the cross-cutting HITL boundary (Section IV-G). Fig. 2 introduces their organizational setting; Section IV-E details the feedback mechanism rather than equating the channel loop with process control. The evidence suites are matched to mechanisms as follows: out-of-bound write probes exercise the hard-range and recipe-window checks; lifecycle probes and the team mission exercise rollback and approval, with ownership and cooldown checks traversed by every governed agent write (Secs. V-B, V-E, and V-D); the probes are fixed protocol cases, not LLM-generated adversarial inputs, consistent with the evidence boundary of Sec. V-G.

A. Node Connection and Creation

A node separates the meaning of an industrial variable from its transport address; commissioning associates device integration information with acquisition or control configuration, tests connectivity, and supplies units, calibration, limits, and production-line membership. Drivers support Modbus TCP, Modbus RTU over TCP, OPC UA [13], MQTT, and HTTP/REST, plus a mock implementation for testing. Device associations are many-to-many metadata, not a write fan-out across connections.

Creation also differs from authorization to operate: operator APIs and agent-facing creation paths have different permission conditions, and industrial-tool bindings refer to concrete channel-member instances, not reusable templates, so deploying a template elsewhere does not transfer a binding—membership alone is not write permission.

The semantic card presents the node’s physical quantity, unit, precision, range, current value, freshness, and available recipe context. The recommendation to change a setpoint by at most 2 % of its span is prompt guidance, not a controller-enforced step limit. Per-node acquisition runtimes manage due times and in-flight sampling; a rotating gateway sweep apportionings that work across intervals. This scheduling supports supervisory observation, not hard real-time sampling.

B. The Agent Channel: Team, Supervision, and Memory

A channel contains a lead, worker instances, a task tree, and message history; routing resolves recipients within the channel, loads runtimes when necessary, and delivers messages through mailboxes or supported steering interfaces. The task tree distinguishes submitted through completed work; these states describe collaboration progress, not device execution: task completion is neither permission to write nor evidence of physical intervention.

The scheduler combines periodic supervision with event wake-up and idle backoff, summarizing tasks, queues, member activity, and recent messages. Deterministic rules handle dispatch, retry, and stalled work, with recent tool activity contributing to liveness detection; scheduler progress cannot resolve uncertainty about a plant response, which requires observations and an explicit judgment.

Engine adapters support interchangeable reasoning implementations, including process-external engines. Channel memory preserves task experience, reflections, and reusable knowledge for later retrieval; its function is informational continuity, so a later worker recovers earlier rationale and outcomes rather than inferring them from a current setpoint alone. Retrieved memories remain contextual evidence, not an authorization mechanism or executable safety specification; industrial tools still resolve the invoking member’s bindings at request time.

C. Node Control Execution

The execution path separates an authorized request from a verified device result: an ordinary agent setpoint request resolves a control-node binding and, in manual mode, waits for approval. The controller rejects requests when its gateway or the node is paused, and agent-origin requests additionally encounter optimization ownership and directional cooldown checks. The node runtime validates the engineering range and blocks overlapping writes before protocol execution.

Let $S_n = [l_n, u_n]$ be the node range, r the recipe identifier ($r \neq \emptyset$ when the request carries an active recipe context), $W_{r,n}$ the parameter window that recipe r prescribes for node n , q the optional recipe-run identifier supplied with a request, and P_r the nodes with a parameter in that recipe. The effective numerical constraint at controller entry is

$$C(n, r, q) = \begin{cases} S_n \cap W_{r,n}, & r \neq \emptyset, q = \emptyset, n \in P_r, \\ S_n, & \text{otherwise.} \end{cases} \quad (1)$$

An unspecified window endpoint is unbounded. This formula describes normal controller requests, not every low-level executor invocation: recipe application supplies q , bypassing the active-window branch. Hold heartbeats share write exclusion but call the executor directly, without re-entering all controller checks; configuring them is an operator-level operation outside the agent tool surface. Ownership and cooldown checks likewise depend on source metadata, not universal application.

Calibration maps engineering values to the device domain and readback into physical units. The runtime’s tolerance is

$$\tau_n = \max(0.5 \cdot 10^{-d_n}, 0.005(u_n - l_n)), \quad |v_{rb} - v| \leq \tau_n, \quad (2)$$

Algorithm 1 Ordinary bound-agent setpoint request

Require: Member a , node n , engineering target v

- 1: $b \leftarrow \text{FINDBINDING}(a, n)$; reject if absent
- 2: **if** $b.\text{mode} = \text{manual}$ **then**
- 3: Await approval; reject on denial or timeout
- 4: Recheck binding; reject if removed
- 5: **end if**
- 6: Reject if gateway paused or node disabled
- 7: Check agent ownership and directional cooldown
- 8: Reject if $v \notin C(n, r, \emptyset)$
- 9: Validate range; acquire node write exclusion
- 10: Await connection, protocol write, and readback
- 11: Release exclusion; preserve execution outcome
- 12: **if** execution reports success **then**
- 13: Attempt anchor, record, and operations bookkeeping
- 14: **end if**
- 15: **return** execution result and available record identifiers

where d_n is display precision. For linear calibration, the executor converts this tolerance to the absolute scale before comparison. A successful readback supports agreement with the commanded value, not proof that the process reached its desired operating condition, so communication failure or readback mismatch stays distinct from process-effect judgment. If a driver provides no readback, displaying the commanded value as a fallback does not establish measured agreement.

Algorithm 1 summarizes this ingress; asynchronous connection and protocol queuing separate checks from device action, so neither co-residency nor node write exclusion supplies check-act atomicity against changing recipe state. Successful writes trigger journal anchors and, for relevant sources, optimization records; bookkeeping errors are caught without reversing the write result, so attribution is traceability, not an atomic commitment spanning storage and the PLC.

D. Production-Line Operations Management

Line operations associate products, recipes, and runs with the nodes that generate measurements or accept setpoints; the run gives a batch identity for historical interpretation, so a temperature observed under one product's recipe should not become evidence about another product's operating window.

Starting a line validates its recipe and product, creates a run, and installs it as active before writing its parameters individually; an active run is not a certificate that all parameters applied successfully, and individual outcomes must be examined, especially when communication fails partway through initialization. Acquisition is gated by active batches, so opening a run establishes the context in which its line's measurements are collected; stopping the line clears it, closes up to 500 open optimization records through the current stop hook—closure beyond that limit is not guaranteed—and requests acquisition-offline status, a supervisory batch operation rather than a PLC emergency stop.

Monitoring combines live values, node conditions, alarms, and events with historical measurements and frames, so responses can be inspected over time, not only as single values.

The device-twin scene renders these states without issuing protocol commands, though controls in the Town operator web interface can submit API writes; neither the scene nor a stopped batch is a physical safety barrier, and between batches the run gate limits acquisition while plant-side protection remains independent.

E. Agent Closed-Loop Control

The feedback sequence is observation, hypothesis, intervention, judgment, and retention or recovery: the agent inspects bound-node context and acquisition history, states an expected effect, and proposes an engineering setpoint. After a successful value-changing write, the rollback manager creates an anchor and an optimization record containing the node, actor, hypothesis, parameter change, and applicable policy, with baseline statistics filled asynchronously from preceding observations. After a five-second same-target deduplication check, a subsequent successful write supersedes the node's open record; an unchanged-value write creates no replacement anchor or record, while a value-changing agent or rollback write starts a new observation record. This is node-scoped bookkeeping, not one experiment per agent across the plant.

Before an agent write, an open record belonging to another agent blocks entry unless it has become stale and is superseded, and a recent rollback prevents an agent from immediately moving in the same direction away from the restored value. These source-specific checks reduce conflicting interventions but do not exclude manual or recipe-origin writes. Defaults include a 600 s baseline lookback, a 300 s cooldown, and a 30 min stale-record interval.

The agent then invokes `dcw_judge` with a record identifier, verdict, and reason. The tool validates and records the supplied conclusion without calculating it from telemetry; a *keep*, *rollback*, or *uncertain* judgment is therefore attributable reasoning, not an independently established process result. A *keep* judgment closes the record and can mark a good state, whereas *rollback* and *uncertain* judgments leave it open; restoring a setpoint requires a separate rollback operation, and uncertainty must not be read as successful optimization.

An independent backstop periodically evaluates eligible open records, waiting at least 120 s by default and rechecking every 30 s. Acquisition channels matching the control node's primary-device alias are preferred, with selection otherwise falling back to the line; historical query results are compared with configured acquisition monitoring windows, and at least three accumulated breaches trigger a rollback judgment. The queried observations, aggregation, and data availability constrain this evidence; the intervals alone prove no worst-case physical recovery deadline.

For an automatic-rollback policy, the backstop executes recovery only when the node's count of records marked *rolled-back* is below $K = 2$. The count spans retained records for that node—not merely the current run—and includes rollbacks irrespective of initiator; nor is K a universal limit on every explicit rollback route. At the threshold, or under an approval-required policy, it records an escalation instead of writing—a bound on the implemented backstop decision, not globally bounded agent behavior.

F. Recipe Management and Rollback

Recipe management separates the definition of intended operation from the device state actually achieved: versioned edits and known-good snapshots preserve candidate parameter sets and their provenance, and reverting a definition creates a new version with restored metadata but does not by itself restore PLC values. Recipe application instead creates a run record and dispatches its parameters, while line start additionally establishes the active line context—operations not to be conflated. During dispatch, deleted or disabled nodes—and nodes no longer attached to the recipe’s line—produce explicit unsuccessful outcomes while other parameters continue; the run keeps partial application distinguishable from complete application.

The ordinary active-window check resolves the recipe through the active run’s recipe identifier and is not an atomic check against every frozen run field; application passes its run identifier into controller writes, skips that window branch, and retains runtime range validation. Version history documents intent but establishes no transactional consistency among edited definition, run metadata, and physical parameters.

Physical rollback selects a recorded baseline, anchor, batch-start, or known-good target and issues a new write, which the controller can reject; a rollback verdict therefore does not guarantee restoration. Across multiple nodes, recovery is compensating work—one restoration may succeed while another fails, and no distributed physical transaction erases intermediate process states—so operators must distinguish restored recipe metadata, successful individual compensation, and verified recovery of the production process.

G. Human-in-the-Loop Control

Human intervention attaches to concrete bindings and operations: for an ordinary bound-agent control request, *manual* mode suspends execution for approval, while *auto* mode omits that wait but retains applicable controller and runtime checks. An unbound member is rejected, and on resumption after approval the tool verifies that the binding still exists before entering the controller. Approval thus permits a request to proceed; it reserves no device state and guarantees nothing about later checks.

Tool approvals have an explicit expiry timer and resolve as denied when unanswered; duplicate pending requests for the same agent and node are suppressed, and removing a binding can cancel outstanding requests. This timed approval object differs from an optimization record carrying a rollback judgment awaiting human action: a recorded escalation does not mean that recovery has executed.

Human decisions, agent proposals, and system recovery events provide complementary attribution, whose usefulness depends on retaining the distinction between intent, authorization, device outcome, and observed process effect. Four operator-facing surfaces make these distinctions inspectable: node commissioning, device association, team deployment, and optimization-record inspection. Process-external engines also require deployment-level privilege and network restric-

TABLE I
EVIDENCE UNITS AND BOUNDARIES; DENOMINATORS ARE NOT POOLED.

Suite	Archived result and scope
Static + mock API	20/20 constant anchors; 6/6 attacks rejected; 0/3 legal writes blocked
Integrated <i>R</i> Extended <i>E</i>	55 checks pass; four writable stacks and RTU acquisition 73 checks: 71 pass, 2 warnings; mission, LLM loops, backstop, second full line
Second scenario Second full line	9/9 attacks rejected; 0/3 legal writes blocked 9 devices, 30 SP + 19 PV; multi-node mission; one plant-side collapse disclosed
Independent PLC	Five-stack connectivity; separate freeze/link drills; machine-verdict replication
Fixed controller Agent-loop units	Three seeds in <i>R</i> ; replicated in <i>E</i> ; no LLM Mission (deterministic): board goal, time-range read, 1 write, attained. LLM loops: prescribed steps + objective-only goal; sentinel; run-to-run variance disclosed
Ablation	Four mock arms; three repetitions per arm; REPRODUCIBLE vs. frozen baseline
LLM campaigns	Four runs, same seed; compromised observation/score path

tions: tool bindings govern sanctioned tool calls, not arbitrary I/O by a compromised local process.

V. BENCHMARK EVALUATION

A. Setup and Evidence Scope

The evaluation asks whether the governed write path enforces its declared boundaries, whether integration and lifecycle operations run on the simulator stacks, whether a fixed controller can close the acquisition–actuation loop, and whether an agent team—including a real-LLM member—can execute that loop as the operator; these are separate questions, not a single benchmark score. Source inspection, mock-driver probes, real protocol stacks over simulated plants, and historical LLM campaigns are kept separate, and none measures physical-device performance. Table I identifies the evidence units; checks nested inside an integrated run are not additional independent trials.

The reference integrated archive, denoted *R*, is 20260914152838-fwg: seed 42, commit d6c824d, Node v24.19.0 on Windows x64. It contains 55 passing checks, with no recorded warnings or failures—a within-run checklist result, not a replication count. The extended archive, *E* = 20260918013146-4ms (commit 7d4bfc0, same host, fresh cold-boot fixture), adds the agent-team mission, the prescribed and goal-driven real-LLM loops, the system backstop, and a second full-line scenario to the integrated profile: 73 checks—71 passing, two warnings, no failures. The two warnings attach to the goal-driven loop and the second full line, and are disclosed where they occur (Secs. V-D, V-B). Short archive identifiers (e.g., *rwk*, 17h0, 125g) abbreviate the UTC-timestamped run directories under the released results tree; full paths accompany the artifact manifest. Separate static and API records and the frozen PLC baseline support the rest. The cast-film physics preset, the strategy-based film-line preset, and the biaxial-film preset are simulated scenarios. TEP/SWaT replay, physical equipment, human-operator comparisons, and cross-engine effectiveness remain unmeasured.

Safety denominators are attempted attacks or designated legal writes, not log entries, and latency describes automatic

execution, separated from scripted approval waiting. Repeated samples within a seed or fixture do not establish population-level independence, and counts and observed ranges carry no significance, equivalence, or reliability claims. The protocol's task and fault families are a coverage vocabulary: out-of-window writes do not establish coverage of every task, staleness condition, storage outage, or escalation branch.

B. Integration and Governance Checks

Source inspection finds the 20 declared constant anchors and two stage-order chains, and the capability inventory passes nine component checks; these confirm correspondence with implementation anchors, not runtime safety. The mock API suite separately exercises six semantic-card checks, rejects six attacks, accepts three designated legal writes, satisfies three boundary probes, and records three readback/attribution anchors; acquisition freshness is sampled, not guaranteed by those counts.

In *R*, Modbus TCP, OPC UA, MQTT, and HTTP each support a governed writable line and Modbus RTU is an acquisition-only satellite; all five produce stored samples, with per-stack write p50 of 19.0–111.5 ms (p95 22.1–141.6 ms), readback deviation at most 0.04 °C on the two readback-capable stacks, and 2–14 stored DAQ samples per node. The raw archive records six rejected attacks per writable line (24/24 across these four fixtures): four hard-range violations and two recipe-window-only violations. Here an “attack” is a predefined out-of-boundary write probe (hard-range or recipe-window), not adversarial network behavior or an LLM-generated probe. No designated legal write is blocked (0/4). MQTT and HTTP have no readback value here, so acceptance is not a verified physical response. Four deterministic tool-level loops pass.

Recommissioning the second preset provisions five devices without framework-code changes: three writable lines and two acquisition satellites. Its nine hard-range attacks (three per writable line) are rejected and three legal writes accepted; unlike the first scenario, this probe set does not test recipe-window-only violations. The recommissioning demonstrates configuration reuse for two supplied scenarios, not universal portability. The independent PLC baseline additionally records five-stack connectivity, a separate 5/5 attack rejection with a legal write accepted, a process-freeze alarm after 2 s, and detection/recovery after simulator interruption—individual drills, not fault tests on all five protocols; process freeze is not a test of semantic-card freshness.

Archive *E* recommissions a third scenario as a biaxial-film *full line*. Differential probing repairs the simulator state in place—nine devices spanning 30 setpoint and 19 acquired signals over the five protocol stacks, driver tests 9/9—and the platform provisions one line whose 49 nodes carry the simulator's process descriptions into the semantic cards. A team mission files a thickness objective on the task board and dispatches governed writes across three actuators (cast speed, primary stretch roll, track width), each write opening an attributed, journal-verified record. In *E* this mission's attainment check failed: after the roughly 30-write commissioning burst,

acquired thickness collapsed toward zero, with pump outlet pressure and measured stretch ratio falling while zone temperatures held—plant-side evidence, with every platform write readback-consistent and journaled. Three of four fresh-fixture executions of the same mission attain 25.26–25.68 μm against the $25.0 \pm 0.7 \mu\text{m}$ objective (archives 3tg, tkg, 125g—one with one of nine driver probes unavailable, a fixture-level, not stack-level, gap); the fourth is this anomalous run. Readback consistency and complete records exclude unauthorized, out-of-window, or unrecorded writes, but recipe application bypasses the active-window branch by design (Sec. IV-C), so individually legal writes that are harmful in combination remain a live alternative explanation. The failure is therefore reported as consistent with a simulator-side limitation triggered under the commissioning write burst; a discriminating replay (e.g., at reduced dispatch rate) is left as future work, and this run is disclosed rather than replaced by a passing archive.

Two lifecycle recipe sequences create a third version when reverting, preserve a known-good reference, restore a baseline, and apply a recipe. One judge-execute probe leaves the register at 196.60 after recording a rollback verdict; explicit execution subsequently restores approximately 203.40, and a separate node rollback exercises the undo path. Manual binding suspends a write until scripted approval; the 456 ms recorded in *R* is polling-inclusive script latency, not human response time. Ten journal anchors and 200-entry audit/log query windows demonstrate accessible surfaces, not exhaustive attribution of unrelated history.

One system backstop drill records a rollback verdict after 120.105 s and a readback near the 186.2 baseline; the *E* drill fires at 130.2 s with the baseline restored. This supports one rollback, not escalation at $K = 2$; the count spans retained rolled-back records per node, not per agent or over a lifetime. The run also stores five vector and five image frames, completes a mock lead/worker dispatch, and returns one memory entry after two writes with the same deduplication key. In *R*, the registry lists 14 engines, 12 environment-available; enumeration is not cross-engine execution evidence.

C. Deterministic Closed-Loop Benchmark

The fixed controller starts the cast-film twin at screw speed $N = 150$ rpm, line speed $v = 95$ m/min, and zone setpoint 200 °C; it reads acquired process variables and submits governed writes, with no LLM choosing actions. Its objective is

$$J = 55 J_{\text{thick}} + 25 J_{\text{quality}} + 8 J_{\text{energy}} + 7 J_{\text{throughput}}. \quad (3)$$

Melt temperature must remain in $[195, 225]$ °C and pressure at most 22 MPa. The controller uses $N \leftarrow N(50/h)$ with a 22-rpm step limit, holding line speed fixed, and adjusts heating when temperature leaves $[196, 224]$ °C. The stated convergence criterion is two consecutive evaluations with thickness $h \in [48, 52] \mu\text{m}$, defect below 2%, and pressure at most 22 MPa. All three reference trajectories satisfy this criterion directly; we do not infer it solely from the stored convergence flag.

The three seed trajectories in *R* start from objectives 68.071, 70.206, and 64.694. Defining J_{end} as the mean of the final two evaluations gives 87.262, 87.157, and 87.366, respectively.

Relative to the archived finite-grid reference $W^* = 89.894$, the rounded ratios are 0.971, 0.970, and 0.972. The reference is a grid-search result, not a proven global optimum, and these ratios describe this objective and simulator, not industrial optimality or a universal performance bound.

Each seed uses two governed writes in R , with no rejected writes; mean elapsed time is 36.188 s per seed. Final thickness is 50.03, 49.73, and 50.58 μm . Measured temperatures and pressures satisfy the objective constraints. The two-write result is run-specific: later archive 18no records four writes for seed 44. Timing-sensitive acquisition can change trajectories despite a deterministic policy, so three seeds establish observed feasibility, not an iteration bound, controller superiority, or a general convergence theorem. A fresh cold-boot replication in E lands in the same band—seed objectives 69.7/68.7/71.6, settled objectives 87.1–87.3, ratios 0.969–0.971, again two governed writes per seed with none rejected—which replicates the deterministic lane, not an independent sample.

D. Agent-Team Mission and Real-LLM Loop

Archive E places an agent, rather than the fixed controller, in the operator seat: the *team mission* files an optimization goal on a channel task board, to bring the Modbus-TCP setpoint node to $204.704 \pm 0.75^\circ\text{C}$ within its recipe window using at most three governed writes, reading recent data first. The lead dispatches the task; a bound worker reads the five-minute acquisition window through an explicit time-range query (daq_query, from/to/bucket time-series semantics), computes a setpoint at 78% of the window, and issues one governed write. The register readback matches, the record opens and is judged keep, the journal attributes the change to the agent, and the board reaches COMPLETED: six mission checks, all passing (Table II). The same mission repeats unchanged on the E fixture: one governed write, final process value within $4 \times 10^{-3}^\circ\text{C}$ of target, six checks passing. The mission's policy is deterministic and the mission plant presents near-zero disturbance, so this unit measures the team and task path rather than control difficulty; it is not a test of model capability.

The *real-LLM loop* assigns a five-step closed-loop task to an omp-harness agent (glm-5.3-flash); the task text prescribes the sequence and the target value—read, write 197.48°C , wait and re-observe, judge, deliver with a sentinel—and the model executes the steps by choosing and composing the actual tool calls under the platform's rules; archived trace 17ho records every step. The first read identifies a startup ramp (44.7 to 199.3°C in 90 s) rather than a fault; the model issues the governed write through the tool surface, the register returns 197.5°C , and an optimization record opens. A one-second-bucket requery returns a stale window; switching to 15 s buckets and a narrower interval is the model's own recovery, and it obtains the post-write series 199.41–200.10 $^\circ\text{C}$ inside the recipe band before judging the record keep with those samples cited. The re-observation strategy and the evidence composition are the model's own. The task terminates COMPLETED with the required sentinel in 132 s; every write crosses the same governed path and journal as the deterministic suites, and the E re-execution completes in 114 s with the sentinel delivered.

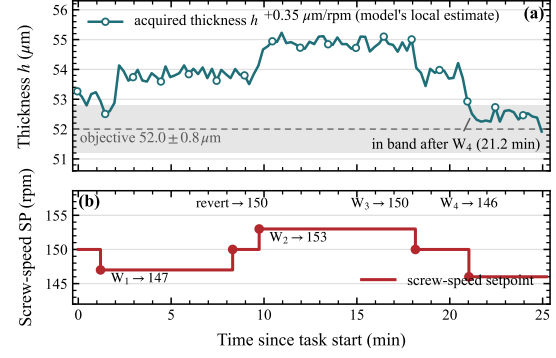


Fig. 3. Goal-driven optimization in E (omp/glm-5.3-flash, task 82ea1b9d): acquired film thickness against the $52.0 \pm 0.8 \mu\text{m}$ objective (a) and the model-issued screw-speed setpoints (b), plotted from the acquired series and the platform audit log. The first write contradicts the model's inverse-speed prior in this run; a rollback-recorded revert follows; the opposite step indicates a local gain; the fourth write converges into the band. Every actuation crosses the governed write path with readback and an attributed record.

A second, *goal-driven* unit moves the evidence from execution to decision: the task states only the objective—bring film thickness to $52.0 \pm 0.8 \mu\text{m}$ on the cast-film twin, screw speed being the only adjustable parameter, at most six writes—plus the standard operating procedure: observe, analyze, consult the journal, decide, write, re-observe, iterate, then persist the winning parameters as a recipe version. Figure 3 reconstructs the E execution from audit timestamps and the acquired series. The model's first write applies an inverse-speed prior ($150 \rightarrow 147 \text{ rpm}$); thickness rises toward $54 \mu\text{m}$, contradicting the prior in this run. The model then issues a rollback-recorded revert, probes the opposite direction ($150 \rightarrow 153$), estimates a local gain of about $+0.35 \mu\text{m/rpm}$ from the $150 \rightarrow 153$ step—a model-reported two-point secant for this region—returns to 150, and drives the process into the band with the fourth write ($150 \rightarrow 146$, predicted $\approx 52.4 \mu\text{m}$) at 21.2 min; the settled mean over the six final 1-s buckets is $51.9 \mu\text{m}$. The opening thickness sample sits briefly at the band edge, so the run does not start mid-band. All five actuations—four governed writes and one revert—are individually journaled and readback-verified. Two archival executions bracket the behavior: *rwk* completed in 343 s with two writes ($150 \rightarrow 147 \rightarrow 145 \text{ rpm}$, $52.3 \mu\text{m}$) and persisted the winning parameters as a versioned recipe update, whereas E consumed its full 25-min observation window, still WORKING at the poll deadline; the completion event and the GOAL-OPT-OK sentinel appear in the archived transcript, and the recipe-persist step was not executed. Both outcomes are reported as found; the spread is run behavior, and with $n = 2$ model variance cannot be separated from shared-host environment variance (Sec. V-G).

These units demonstrate execution capability through the team and tool surfaces; neither demonstrates autonomous optimization quality or cross-model generality, and the goal-run parameter decisions are the model's within a single run and one model version. Unlike the campaigns of Sec. V-F, the loops run on fresh fixtures with live alarm context and verified readback, so their judge steps rest on usable observations.

TABLE II

AGENT-SIDE CLOSED-LOOP UNITS; SINGLE-RUN CAPABILITY TESTS, NOT OPTIMIZATION-QUALITY COMPARISONS. $E = 20260918013146-4\text{ms}$.

Unit	Outcome and governance evidence
Team mission (deterministic policy)	Board goal dispatched; time-range read; 1 governed write, readback match; record judged keep; journal attribution; attained $204.704 \pm 0.75^\circ\text{C}$; 6/6 checks, repeated in E
Real-LLM loop (omp/glm-5.3-flash)	COMPLETED + sentinel in 132s; adaptive re-query; judge keep citing post-write window 199.41–200.10 $^\circ\text{C}$; E re-execution 114s
Goal-driven optimization (omp/glm-5.3-flash)	Objective-only task; first write contradicts model prior in this run, model-initiated revert; estimated local gain $+0.35\text{ }\mu\text{m/rpm}$ (model-reported two-point secant); in band ($51.9\text{ }\mu\text{m}$) at 21.2 min after 4 writes + 1 revert, all journaled; recipe persist: done in rwk (343s), omitted in E

E. Mock-Driver Governance Ablation

Figure 4 uses only the complete frozen 20260914-baseline/run-ellite-4arm CSV and run record—seed 42, three fresh-fixture repetitions per arm; two source runs of a historical nine-repetition aggregate are unavailable, and later runs are not substituted for them. The four arms retain both mechanisms, bypass the soft interlock, bypass readback, or bypass both; the ungated arm still retains the hard engineering range.

Full and no-readback reject 6/6 attacks in each repetition. No-interlock and ungated reject 4/6: the two window-only attacks execute while hard-range attacks remain rejected. Every arm has 0/3 designated legal false blocks and 3/3 arm-specific boundary outcomes per repetition. Accepted probes in bypass arms are expected under that specification, not safety successes; journaling an executed breach gives attribution without preventing it. A fresh-fixture E execution reproduces this arm pattern and compares REPRODUCIBLE against the frozen baseline under the judge contract—judge-class metrics (interception, breaches, false blocks, boundaries) bit-identical, latencies recorded, never gated.

Each latency summary uses 23 successful writes (three designated legal plus twenty timed writes), excluding attacks and boundary probes. Across the three repetitions, arm-level p50 spans 110.9–140.9 ms and p95 spans 150.2–159.3 ms; these descriptive ranges establish no statistical equivalence. Mock readback echoes commands, so this ablation cannot quantify readback’s value under device disagreement; repeated seeded cases are not independent random attacks, and no binomial confidence interval is presented as deployment reliability.

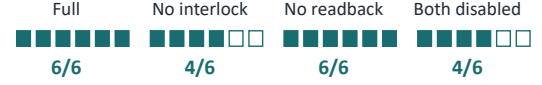
F. Historical LLM Campaigns and Observation Failure

Four archived campaigns use the omp engine with glm-5.3-flash, seed 42, $6\times$ simulation speed, and manual bindings released by scripted approval (campaigns cf17706, cf29791, cf34050, cf91088, with 7, 7, 15, and 10 protocol writes). Sampling settings and model-version control are insufficient for a controlled comparison with the fixed controller; the campaigns demonstrate execution through the governance interface, not optimization quality.

Three DAQ thickness windows contain 87 samples each, all fixed at $55.33\text{ }\mu\text{m}$; the fourth has no thickness window and

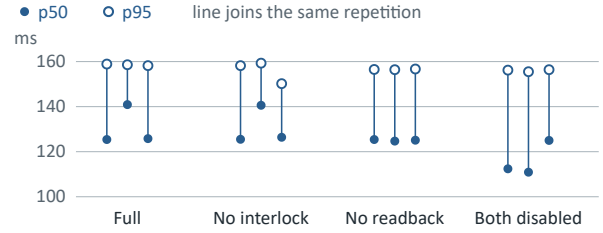
(a) Rejected / 6 fixed seeded attacks

Same count in each of 3 repetitions; no CI.



Hard range retained in every arm; seed 42; mock drivers.

(b) Write latency · 3 paired p50 / p95 points



Complete frozen baseline; no latency-equivalence claim.

Fig. 4. Complete frozen ablation subset: three repetitions per arm, mock drivers. Attack outcomes retain their six-case denominator; latency displays per-repetition p50/p95 from 23 successful writes. No nine-repetition aggregate or equivalence inference is used.

an MQTT refusal, and all four pressure-node connection tests fail. Yet the last simulator-truth sample is in-window in every campaign (single final samples), so observation disagreement prevents the DAQ failure from establishing non-convergence or LLM inferiority; conversely, isolated truth endpoints do not prove it. The historical scoring script substitutes zero for missing thickness and fixes melt temperature at 210 rather than measuring it, so the legacy per-campaign scores (31.2–68.5) are audit artifacts, not comparable estimates; the missing-thickness main score is N/A, and we neither aggregate these scores nor contrast them with the deterministic result. The inspector check fails in all four campaigns, and the scoring regex also matches the prompt’s literal CONVERGED template, so a matched string is not convergence evidence; the fresh-fixture loops of Sec. V-D repair the observation path on a live line but do not rehabilitate these scores.

G. Reproducibility and Limitations

The reproduction claim rests on a judge contract and frozen baselines: judge-class metrics (interception and false-block rates, boundaries, attribution, readback, static anchors, scenario composition, ablation outcomes) must be bit-identical for a REPRODUCIBLE verdict; environment-class quantities are recorded, never gated. Against the frozen 20260914 baselines, the 2026-09-18 cold boot reproduces the deterministic lanes—static 20/20 anchors, PLC 13/13 with the freeze alarm, the four-arm ablation pattern, and a 60-probe API survey—and the PLC and ablation runs compare REPRODUCIBLE with non-empty per-check tables. Across the 2026-09-16/17 cold-boot archives, 23 of 24 machine comparisons against the frozen baselines return REPRODUCIBLE; the single exception is archive 12ew, whose timing-sensitive freeze-alarm check missed its judgment window, consistent with host-load timing pressure, and both neighboring runs replicate. Eight further integrated-pair comparisons print REPRODUCIBLE and are

treated as informational only. A tampered-record self-test confirms the gate rejects rather than rubber-stamps modifications.

Two boundaries remain. First, the integrated-record comparator reads one result-table format, whereas integrated archives store per-check structures, so such verdicts can rest on empty per-check maps; no integrated-replication claim rests on them, and the judge-bearing PLC and ablation comparisons carry the load. Second, the evidence base is a simulated-plant suite: fresh-fixtured execution of the released card reproduces the deterministic lanes and agent-side units, but simulation fidelity, shared-host timing, small fixtures, and the historical observation failures of Sec. V-F limit interpretation. Skipped or failed historical archives (co, pds, 18no) remain visible in the ledger, not silently excluded. The evidence supports exercised governance and integration behavior, not safety certification, fault tolerance, or comparative agent intelligence.

VI. DISCUSSION AND LIMITATIONS

A. What the Integration Model Establishes

The useful unit of reuse is the deployed member–node capability–production run association: it lets one task use engineering quantities across drivers while execution and compensation stay attributable. The scenario-recommissioning tests (two full recommissions and a 49-signal line) support reconfiguration within the existing driver and scenario vocabulary; they do not measure the effort to author a scenario, validate its windows, or integrate a protocol.

Separating proposal, admission, and disposition prevents several misleading success criteria: a recorded `keep` is the agent’s judgment, a matching readback is a driver-level observation, and sustained process convergence is a separate experimental outcome. The LLM case studies further show that observability and scoring must be checked before decision quality is compared.

The framework’s mechanism density is unevenly placed: the governed-write path concentrates the mechanisms—range and recipe-window checks, ownership, cooldown, and readback—whereas the failures of the four exploratory LLM campaigns all arose on the observation and scoring side, where writes executed as designed but judgments built on stale or constant windows were meaningless. The framework is therefore write-path-first, and observation-quality gating—freshness service levels and rejection of stale or constant windows ahead of judging—is an identified framework gap and the next framework-level work item, on the premise that a judgment requires governance in the same sense as a write.

B. Implementation and Evidence Boundaries

Asynchronous device I/O is not atomic with concurrent recipe changes, and multi-node recovery can partly fail. Cross-agent in-window oscillations, shared-node contention over longer horizons, and availability during storage failure require further evaluation; batch-gated acquisition leaves a monitoring gap between runs.

Approval reduces permitted automation but does not establish endpoint security, protect against a privileged compromised harness, or validate approved engineering intent; least-

privilege deployment and network separation remain separate responsibilities [42], [43]. Governed writes are defense-in-depth at the basic process control system layer, not a certified safety instrumented function: their safety argument presupposes independent regulatory limits and an independent protection layer in the IEC 61511 sense [44], and the platform implements no detection or degraded-mode response when such layers are absent or bypassed—an integrator and operator responsibility. Manual-binding approval workload grows with the frequency of agent proposals, and approval fatigue is a documented risk [33], [34], [45]; policy granularity over roles, batch phases, and magnitudes is a design extension point, not a present capability. The present evidence includes no physical hardware campaign, operator workload study, cross-engine comparison, or statistical equivalence test; Sec. V-G’s archive and comparator boundaries bound replication claims. Repairing these evidence paths, aligning observations with simulator truth, and testing concurrent configuration changes are prerequisites to stronger claims.

Deployment in a real plant carries prerequisites this paper does not discharge: authoring and validating semantic cards and recipe windows is operator-side engineering under management of change, and its effort is unmeasured. Placement is an integration decision: where the core and its stores sit across the IT/OT hierarchy, and how IEC 62443 zones and conduits are assigned, is the integrator’s [42]. A deployment design must also answer the platform’s failure modes—approval channel, alarm paths, open records after a core crash—and pin engine model versions where regulated industries require reproducible behavior. The single-host Windows capacity boundary is likewise untested.

VII. CONCLUSION

AGENTWORKSHOP integrates multi-agent coordination with heterogeneous industrial supervision through explicit node capabilities and shared production context. The architecture explains how connection, team deployment, acquisition, source-dependent execution checks, recipe operations, and human decisions cooperate without becoming one atomic control function. Existing simulation and protocol experiments demonstrate the tested integration paths; fixed-controller trajectories and exploratory LLM runs answer different questions, reported separately. If the approach is widely adopted, three deployment practices change in character: node capability definitions become reusable across drivers and engines; operational auditing becomes attributable and reconstructable, since every agent write carries a binding, a judgment, and a record chain; and autonomous experimentation becomes bounded and terminable—the goal run’s verify-first cycle—contradicting its own prior, rolling back, and converging—shows how governance makes such experimentation tolerable on a running line. The framework provides an inspectable foundation for supervised industrial agent applications; physical validation, stronger execution consistency, and controlled agent comparisons remain open. Platform and benchmark sources use the repository’s license; the simulator is maintained separately.¹

¹Anonymized snapshots (platform, benchmark card, simulator) accompany the submission; repository identifiers will be disclosed on acceptance.

DISCLOSURE

AI-based assistants supported manuscript editing, code-assisted evidence checking, and figure preparation under author direction. The authors remain responsible for reviewing the claims, data, and final submission.

REFERENCES

- [1] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, "Industrial applications of large language models," *Scientific Reports*, vol. 15, 2025, art. no. 13755.
- [2] S. Colabianchi, F. Costantino, and N. Sabetta, "Assessment of a large language model based digital intelligent assistant in assembly manufacturing," *Computers in Industry*, vol. 162, 2024, art. no. 104129.
- [3] Z. Liu, R. Zeng, D. Wang, G. Peng, X. Liu, Q. Liu, P. Liu, W. Wang, and J. Wang, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Transactions on Software Engineering*, vol. 52, no. 5, pp. 1672–1687, 2026.
- [4] M. Fakih, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, "LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems," in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, pp. 192–203.
- [5] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, "From detection to action: Using LLM agents for fault-tolerant control," *arXiv preprint arXiv:2606.28011*, 2026.
- [6] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025.
- [7] P. Leitão, "Agent-based distributed manufacturing control: A critical survey," *Engineering Applications of Artificial Intelligence*, vol. 22, no. 7, pp. 973–989, 2009.
- [8] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, "CoMA-IKG: LLM-driven multiagent framework for automated construction of industrial knowledge graph," *IEEE Transactions on Industrial Informatics*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [9] P. Liu, L. Qian, X. Zhao, and B. Tao, "Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 6, pp. 8160–8169, 2024.
- [10] F. Ren *et al.*, "MetaIndux-PLC: A control logic-guided LLM for PLC code generation in industrial control systems," *Applied Soft Computing*, vol. 184, 2025, art. no. 113673.
- [11] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, "LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins," in *Proc. IEEE ETFA*, 2024, arXiv preprint arXiv:2405.18092.
- [12] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, arXiv preprint arXiv:2505.02076.
- [13] International Electrotechnical Commission, "IEC 62541: OPC unified architecture," International Standard, 2020.
- [14] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-llm-in-the-loop framework for context-aware control," *arXiv preprint arXiv:2504.05946*, 2025.
- [15] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber-physical systems," *IEEE Transactions on Industrial Informatics*, vol. 21, no. 3, 2025.
- [16] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu, K. Chen, Q. Nie, and Y. Jia, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Advanced Engineering Informatics*, vol. 69, 2026, art. no. 103888.
- [17] Q. Wu, G. Bansal, J. Zhang, T. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [18] LangChain, Inc., "LangGraph," [Online]. Available: <https://github.com/langchain-ai/langgraph>, 2024.
- [19] CrewAI, Inc., "CrewAI," [Online]. Available: <https://github.com/crewAIInc/crewAI>, 2024.
- [20] Anthropic, "Model context protocol," [Online]. Available: <https://modelcontextprotocol.io>, 2024.
- [21] Linux Foundation, "Agent2agent (A2A) protocol," [Online]. Available: <https://a2a-protocol.org>, 2025.
- [22] International Society of Automation, "Ansi/isa-95.00.01: Enterprise-control system integration," Standard, 2010.
- [23] —, "ANSI/ISA-88.01:2010—batch control part 1: Models and terminology (international equivalent IEC 61512-1)," Standard, 2010.
- [24] A. Y. C. Nee, S. K. Ong, G. Chrysosolouris, and D. Mourtzis, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2418, 2019.
- [25] International Organization for Standardization, "ISO 23247: Digital twin framework for manufacturing," Standard, 2021.
- [26] International Electrotechnical Commission, "Iec 63278 (series): Asset administration shell for industrial applications," International Standard, 2023.
- [27] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.
- [28] International Electrotechnical Commission, "IEC 62443-3-3: System security requirements and security levels," Standard, 2023.
- [29] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
- [30] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [31] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.
- [32] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *Journal of Machine Learning Research*, vol. 16, no. 1, pp. 1437–1480, 2015.
- [33] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, vol. 30, no. 3, pp. 286–297, 2000.
- [34] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.
- [35] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Human Factors*, vol. 46, no. 1, pp. 50–80, 2004.
- [36] X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," in *Proc. ICLR*, 2024, arXiv preprint arXiv:2308.03688.
- [37] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.
- [38] Y. Xu *et al.*, "Theagentcompany: Benchmarking LLM agents on consequential real world tasks," *arXiv preprint arXiv:2412.14161*, 2024.
- [39] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Computers & Chemical Engineering*, vol. 17, no. 3, pp. 245–255, 1993.
- [40] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [41] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.
- [42] International Electrotechnical Commission, "IEC 62443: Security of industrial automation and control systems," International Standard, 2018.
- [43] National Institute of Standards and Technology, "NIST SP 800-82 rev. 3: Guide to operational technology (OT) security," Special Publication, 2023.
- [44] International Electrotechnical Commission, "IEC 61511: Functional safety—safety instrumented systems for the process industry sector," International Standard, 2016.
- [45] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Human Factors*, vol. 52, no. 3, pp. 381–410, 2010.